Design and Analysis of Computer Algorithms

01:198:344

Spring 2026, Rutgers

Instructor: Surya Teja Gavva

Dynamic Programming Overview

- Dynamic programming follows a structured pattern:
  1. Write a recursive formula for the problem.
  2. Compute the formula using a recursive function.
  3. Use a table to store results of subproblems and avoid recomputation.
- Two approaches to dynamic programming:
  - **Memoization:** Store results as you recursively compute them.
  - **Bottom-Up:** Iteratively fill a table starting from the base cases.

Step 1: Recursive Formulation

DP is not about filling tables; it's about smart recursion!

- **Specification:** Describe *what* you want to solve in precise English. (The "Goal")
- **Solution:** Write a clear recursive formula in terms of smaller subproblems. (The "Logic")

This is the hard part. The rest is mechanical.

Step 2: Bottom-Up Construction

Build the solution from the base cases up.

1. **Subproblems:** Identify all distinct ways the algorithm calls itself.
2. **Data Structure:** Choose a structure (usually a multidimensional array) to store every solution.
3. **Dependencies:** Map which subproblems depend on others.
4. **Evaluation Order:** Order subproblems so they come *after* their dependencies.
5. **Algorithm:** Write nested loops and replace recursion with array lookups.

Max Profit

- **Specification:** Let $M[i]$ be the **minimum price** from day 1 to i.
- **Recursive Solution:**

$$M[i] = \min(P[i], M[i-1])$$

- **Base Case:** $M[1] = P[1]$.
- **Goal:** Find $\max(P[i] - M[i])$ for all $i \in \{1 \ldots n\}$.

Max Profit: Stage 2 (DP Table)

- **Subproblems:** n distinct values of $M[i]$.
- **Data Structure:** An array `M[1..n]` to store minimums.
- **Dependencies:** $M[i]$ depends only on $M[i-1]$.
- **Algorithm:**

1. Create array `M` of size n.
2. `M[1] = P[1]`
3. For $i = 2$ to n: `M[i] = min(P[i], M[i-1])`
4. Return $\max_i(P[i] - M[i])$

Max Profit: Space Optimization

- **Observation:** To compute $M[i]$, we only ever look at $M[i-1]$. We never look at $M[i-2]$ or earlier.
- **Optimization:** We don't need the whole array `M`. We can just keep variables `min_price`, `max_profit`.
-
  1. Initialize `min_price` to ∞.
  2. Loop i from 1 to n:
  3. `min_price = min(min_price, P[i])`
  4. `max_profit = max(max_profit, P[i] - min_price)`
- **Space Complexity:** Reduced from $O(n)$ to $O(1)$.
- **Time Complexity:** Remains $O(n)$.

Min Path Sum

Minimum Path Sum

Given a grid where each cell contains a non-negative integer, find a path from the top-left to the bottom-right cell that minimizes the sum of all numbers along its path.

- **Specification:** $Cost(i, j)$ is the minimum cost to reach cell (i, j) from $(0, 0)$.
- **Recursive Solution:** To reach (i, j), you must come from the top $(i-1, j)$ or the left $(i, j-1)$.

$$Cost(i, j) = \text{Grid}[i][j] + \min(Cost(i-1, j), Cost(i, j-1))$$

- **Base Case:** $Cost(0, 0) = \text{Grid}[0][0]$.

Min Path Sum

- **Data Structure:** A 2D array `dp[m][n]`.
- **Dependencies:** Cell (i, j) depends on its top and left neighbors.
- **Evaluation Order:** Row-by-row (top-to-bottom) or column-by-column (left-to-right).
- **Complexity:**
  - **Time:** $O(m \times n)$ (one pass through the grid).
  - **Space:** $O(m \times n)$ (can be optimized to $O(n)$).

Introduction to Greedy Algorithms

- **Definition**: Greedy algorithms make locally optimal choices at each step to find a globally optimal solution.
- **Contrast with Dynamic Programming**:
  - **Dynamic Programming**: Explores all options and stores intermediate results.
  - **Greedy Algorithms**: Select options directly without exploring every possibility.

When Do Greedy Algorithms Work?

- Greedy algorithms are often incorrect and only work in specific cases.
- The goal is to determine when a greedy approach applies and how to prove its correctness.

Knapsack Problem with Unit Weights

Problem Definition:

- **Input**: A collection of n items, each with a weight of 1 and a positive value v_i. Knapsack size W.
- **Output**: Maximum value by picking a subset of items with total weight $\leq W$.

Dynamic Programming Solution: $O(nW)$ time, which can be simplified with certain assumptions.

Greedy Algorithm

Algorithm:

1. Sort items in non-increasing order of their values v_i.
2. Pick the top W items based on sorted order.

Observation: If $v_i > v_j$, then item i is better than j (both have weight 1).

Proof of Correctness via Exchange Argument

- Let $G = \{g_1, g_2, \ldots, g_W\}$ be the greedy solution.
- Let $O = \{o_1, o_2, \ldots, o_W\}$ be any optimal solution, sorted by value.

Exchange Argument:

- Identify the first difference: $g_j \neq o_j$.
- Replace o_j with g_j to form a new solution O'. This does not decrease the solution value.
- Repeat for all items until $O = G$, proving G is optimal.

Runtime Analysis

Sorting Step: $O(n \log n)$ using a fast sorting algorithm.

Selection Step: $O(n)$ for choosing top W items.

Total Runtime: $O(n \log n)$

General Pattern of Greedy Algorithms

1. Make an observation that reduces options.
2. Design the algorithm based on this observation.
3. Prove correctness, usually with an exchange argument:
   - Fix an optimal solution different from the greedy solution.
   - Identify the first difference and exchange it without reducing value.
   - Repeat until the solutions are identical.
4. If proof fails, consider other approaches, such as dynamic programming.

Greedy Algorithms

- Greedy algorithms provide efficient solutions to specific problems.
- They work only when a locally optimal choice leads to a globally optimal solution.
- Always prove correctness before trusting a greedy algorithm!

The Magnetic Tape Storage Problem

Problem Definition:

- We have n files with lengths $L[1 \ldots n]$.
- To read file k, we must fast-forward past the first $k-1$ files.
- The cost of accessing file k is:

$$\text{cost}(k) = \sum_{i=1}^{k} L[i]$$

- Goal: Minimize the expected cost to access a file.

$$E[\text{cost}] = \frac{1}{n} \sum_{k=1}^{n} \sum_{i=1}^{k} L[i]$$

Greedy Algorithm for Magnetic Tape Storage

Intuition: Place shorter files earlier to minimize the accumulated access cost.

Algorithm:

1. Sort files by increasing length $L[i]$.
2. Store files in this sorted order.

Runtime: Sorting takes $O(n \log n)$.

Proof of Correctness: Minimizing Expected Cost

Lemma: $E[\text{cost}(\pi)]$ is minimized when $L[\pi(i)] \leq L[\pi(i+1)]$ for all i.

Proof Outline:

- Assume $L[\pi(i)] > L[\pi(i+1)]$ for some index i.
- Swap files at positions i and $i+1$:
  - Increases cost of accessing file a by $L[b]$.
  - Decreases cost of accessing file b by $L[a]$.
- Total change in cost: $\frac{(L[b]-L[a])}{n}$, which is negative since $L[b] < L[a]$.
- Thus, if files are out of order, swapping reduces the cost.

Generalization with Access Frequencies

Extended Problem Definition:

- Each file i has an access frequency $F[i]$.
- Total cost of accessing all files on the tape:

$$\Sigma\text{cost}(\pi) = \sum_{k=1}^{n} \left(F[\pi(k)] \sum_{i=1}^{k} L[\pi(i)] \right)$$

- Goal: Minimize $\Sigma\text{cost}(\pi)$.

Greedy Algorithm with Varying Frequencies

Intuition:

- Place files with the lowest L/F ratio first to minimize weighted cost.

Algorithm:

1. Sort files by increasing ratio $\frac{L[i]}{F[i]}$.
2. Store files in this sorted order.

Runtime: Sorting takes $O(n \log n)$.

Proof of Correctness:

Lemma: $\Sigma\text{cost}(\pi)$ is minimized when $\frac{L[\pi(i)]}{F[\pi(i)]} \leq \frac{L[\pi(i+1)]}{F[\pi(i+1)]}$ for all i.

Proof Outline:

- Assume $\frac{L[\pi(i)]}{F[\pi(i)]} > \frac{L[\pi(i+1)]}{F[\pi(i+1)]}$ for some index i.
- Let $a = \pi(i)$ and $b = \pi(i+1)$.
- Swapping changes cost by:

$$L[b] \cdot F[a] - L[a] \cdot F[b] < 0$$

  since $\frac{L[a]}{F[a]} > \frac{L[b]}{F[b]}$.
- Swapping reduces the total cost, so files must be in increasing order of L/F.

Scheduling Classes

Problem Definition:

- Given n class time slots represented by intervals $[a_i, b_i]$, where a_i is the start time and b_i is the end time of the class.
- Goal: Schedule the maximum number of non-overlapping classes.

Example:

- Consider classes with time slots: $[9:00, 10:30]$, $[10:00, 11:00]$, $[11:00, 12:00]$, etc.
- A valid set of non-overlapping classes might be $[9:00, 10:30]$ and $[11:00, 12:00]$.

Greedy Algorithm for Scheduling Classes

Algorithm:

1. Sort class intervals by their end times b_i in non-decreasing order.
2. Initialize the schedule with the first class.
3. For each subsequent class, add it to the schedule if it does not overlap with the last added class.

Proof of Correctness for Scheduling Classes

Proof Outline:

- Let G be the greedy solution, O an optimal solution.
- Identify the first index j where G and O differ.
- Swap $O(j)$ with $G(j)$ to create a new optimal solution O'.
- Repeat this process until O' matches G, showing that G is indeed optimal.

Conclusion:

- The greedy solution G schedules the maximum number of non-overlapping classes.

Runtime Analysis for Scheduling Classes

Sorting Step: $O(n \log n)$ for sorting class intervals by their end times.

Selection Step: $O(n)$ to iterate over intervals and select non-overlapping classes.

Total Runtime: $O(n \log n)$

Fractional Knapsack Problem

Problem Definition:

- Given n items, each with a weight w_i and value v_i.
- A knapsack with a maximum weight capacity W.
- Goal: Maximize the total value by selecting items or fractions of items to fill the knapsack such that:

$$\sum_{i=1}^{n} p_i v_i$$

is maximized, subject to:

$$\sum_{i=1}^{n} p_i w_i \leq W$$

where $0 \leq p_i \leq 1$ represents the fraction of item i taken.

Fractional Knapsack Example

Example:

- Knapsack Capacity: $W = 10$
- Items:
  - Item 1: $w_1 = 4$, $v_1 = 2$, $\frac{v_1}{w_1} = 0.5$
  - Item 2: $w_2 = 5$, $v_2 = 3$, $\frac{v_2}{w_2} = 0.6$
  - Item 3: $w_3 = 7$, $v_3 = 4$, $\frac{v_3}{w_3} = 0.57$
- Optimal Solution:
  - Take all of Item 2 for a value of 3.
  - Take $\frac{5}{7}$ of Item 3, contributing additional value:

$$\frac{5}{7} \times 4 = \frac{20}{7} \approx 2.86$$

  - Total value achieved: $3 + \frac{20}{7} = \frac{41}{7} \approx 5.86$, which is greater than the maximum achievable in the 0/1 knapsack.

Greedy Algorithm for Fractional Knapsack

Intuition:

- To maximize value, prioritize items with the highest value-to-weight ratio $\frac{v_i}{w_i}$.

Algorithm:

1. Calculate the value-to-weight ratio $\frac{v_i}{w_i}$ for each item.
2. Sort items by decreasing value-to-weight ratio.
3. Initialize the knapsack's current weight as 0.
4. For each item, add it fully if it fits, or add a fraction of it to fill the knapsack to capacity.

Runtime: Sorting takes $O(n \log n)$.

Proof of Correctness for Fractional Knapsack

- Let ALG be the solution from the greedy algorithm and assume it is not optimal.
- Let OPT be the optimal solution such that:

$$\sum_{i=1}^{n} p_i v_i < \sum_{i=1}^{n} q_i v_i$$

- Assume items are sorted in decreasing order of $\frac{v_i}{w_i}$, and p_i and q_i are the fractions taken in ALG and OPT, respectively.
- Let i be the first index where $p_i \neq q_i$; thus $p_i > q_i$. Then there must exist $j > i$ such that $p_j < q_j$.

Constructing a Better Solution than OPT

New Solution q':

- Construct q' by setting $q'_i = q_i + \epsilon$ and $q'_j = q_j - \epsilon \frac{w_i}{w_j}$.
- The total weight remains unchanged, but the total value increases by:

$$\epsilon v_i - \epsilon \frac{w_i}{w_j} v_j$$

- Since $\frac{v_i}{w_i} > \frac{v_j}{w_j}$, this value is positive, contradicting the optimality of OPT.

Conclusion: Therefore, ALG is optimal.

Runtime Analysis for Fractional Knapsack

Sorting Step: $O(n \log n)$ to sort items by value-to-weight ratio.

Selection Step: $O(n)$ to iterate over the items and fill the knapsack.

Total Runtime: $O(n \log n)$

Introduction to Encoding

What is Encoding?

- Encoding is the process of converting data (like characters) into a specific format, often for transmission or storage.
- Different types of encoding can be more or less efficient depending on the data and the requirements.

Fixed-Length vs. Variable-Length Encoding

Fixed-Length Encoding:

- Each character is assigned a binary code of the same length.
- Simple and easy to implement, but not efficient for characters with different frequencies.
- Example: ASCII encoding assigns 8 bits to each character.

Variable-Length Encoding:

- Characters are assigned binary codes of varying lengths.
- Frequently occurring characters get shorter codes, while less frequent characters get longer codes.
- Can significantly reduce the total bit length required to encode a message.

Prefix Codes

Definition:

- A **prefix code** is a type of variable-length code where no code word is a prefix of any other code word.
- This ensures that the encoded message is uniquely decodable, avoiding ambiguity.

Example:

- Code: $A = 0$, $B = 10$, $C = 110$, $D = 111$.
- This is a prefix code because no code word is the prefix of another.

Non-Example:

- Code: $A = 0$, $B = 01$, $C = 010$.
- Not a prefix code because "0" is a prefix of both "01" and "010."

Introduction to Huffman Encoding

What is Huffman Encoding?

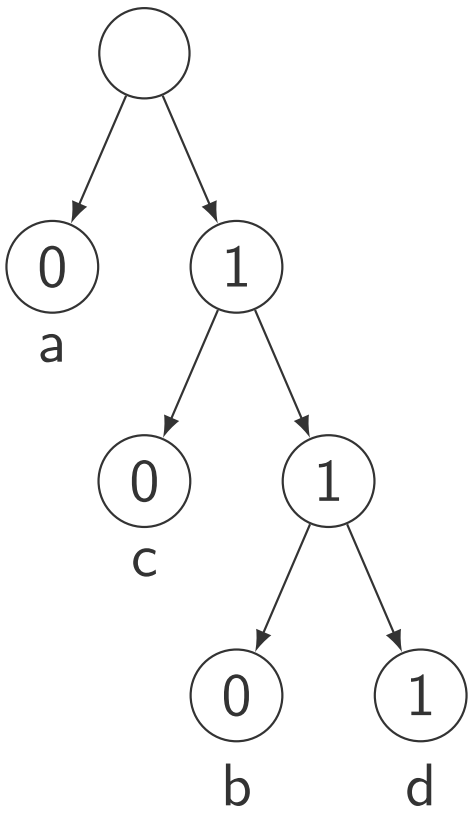
- Huffman Encoding is an algorithm that constructs an optimal prefix code based on character frequencies.
- Frequently occurring characters are assigned shorter codes, while less frequent characters get longer codes.

Prefix Tree for Character Codes

Codewords:

Character	a	b	c	d
Codeword	0	110	10	111

Prefix Tree:



The Huffman Tree Problem

Objective:

- Given n characters $\{c_1, c_2, \ldots, c_n\}$ with frequencies $\{f_1, f_2, \ldots, f_n\}$.
- Construct a prefix tree to assign each c_i a unique codeword.
- Minimize the total cost:

$$C = \sum_{i=1}^{n} f_i \cdot d_i$$

where d_i is the depth of c_i's codeword in the tree.

Constraints:

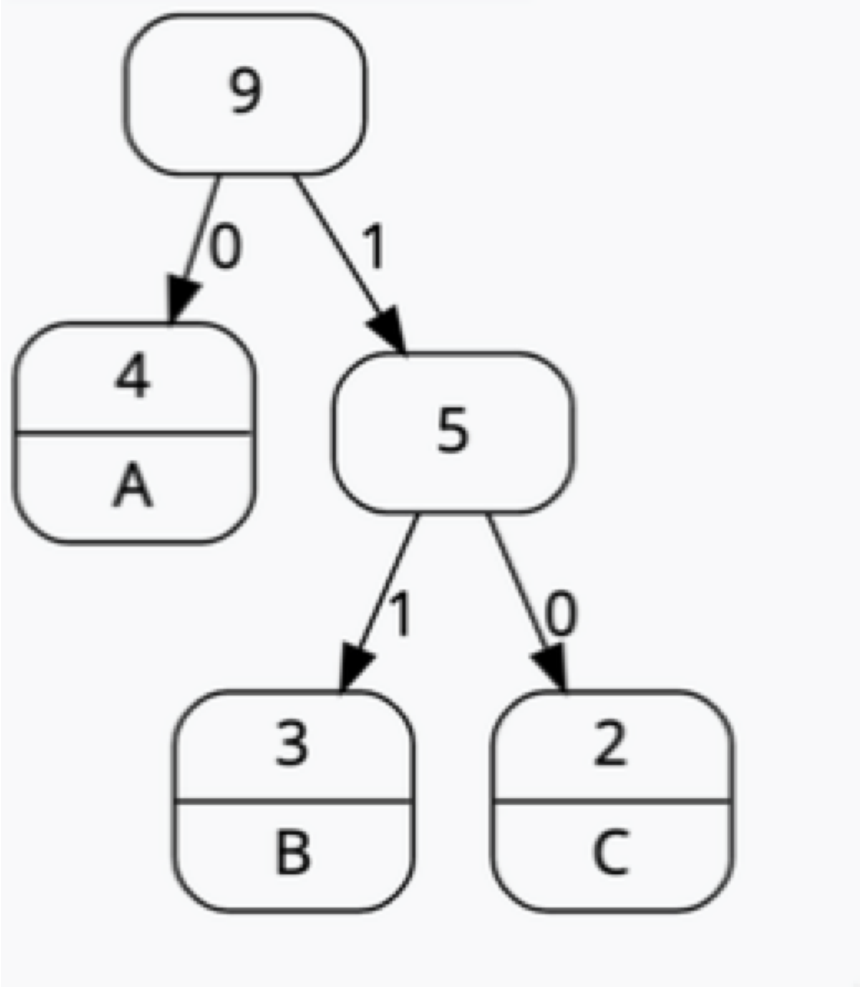
- Prefix-free code: No codeword is the prefix of another.
- Higher $f_i \Rightarrow$ shorter codeword (lower d_i).

Huffman's Algorithm

Merge the two least frequent letters and recurse.

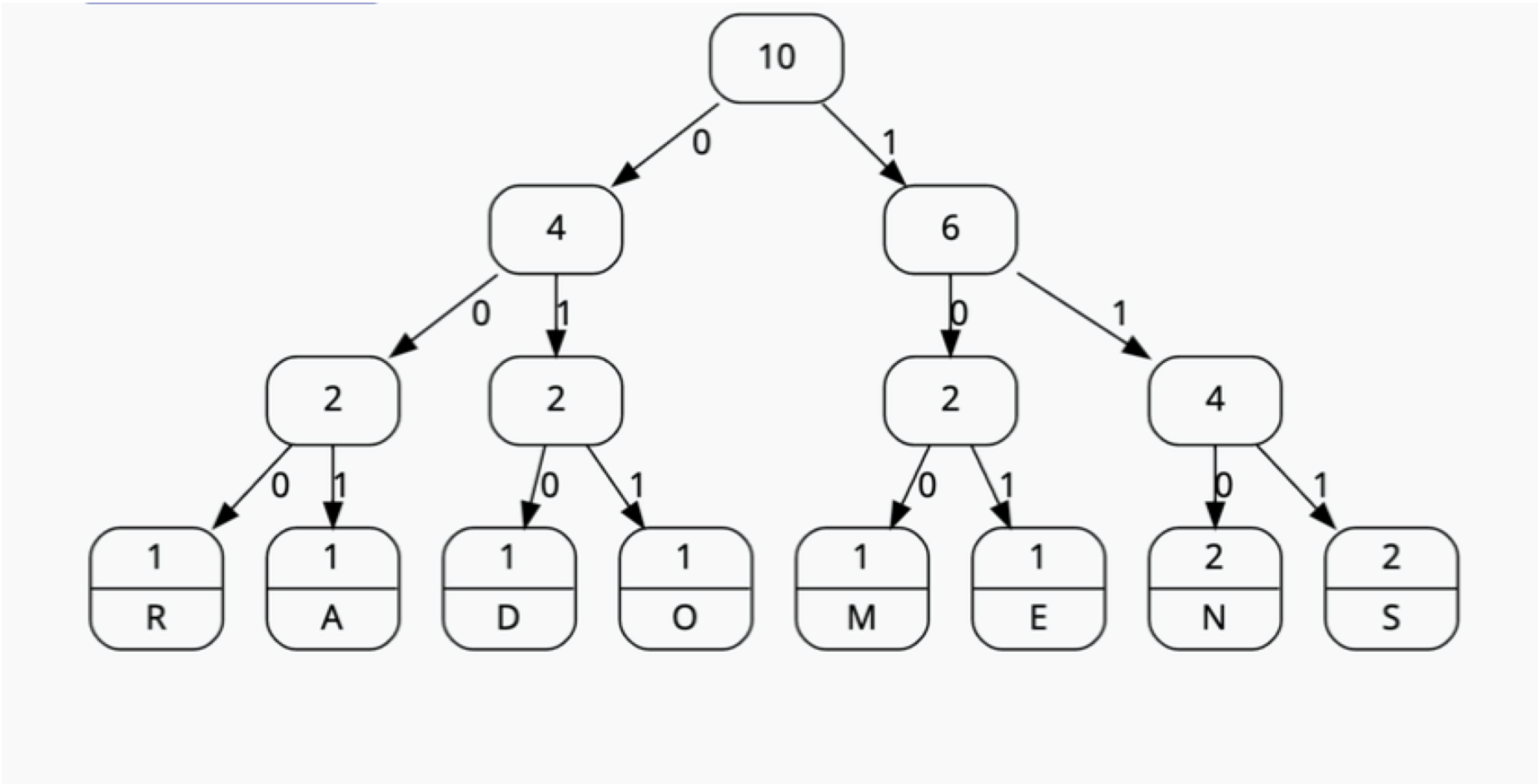
Example

AABCABBCA



Example

RANDOMNESS



Huffman's Algorithm

Pseudocode: `huffman(C, freq)`:

1. Initialize a priority queue Q and insert each character $x \in C$ with key freq$[x]$.
2. **for** $i = 1$ to $|C| - 1$:
   - $z \leftarrow$ new internal node
   - $x \leftarrow$ `extract-min` from Q
   - $y \leftarrow$ `extract-min` from Q
   - left$[z] \leftarrow x$, right$[z] \leftarrow y$
   - freq$[z] \leftarrow$ freq$[x]$ + freq$[y]$
   - `insert` z into Q
3. Return the last element in Q as the root of the Huffman Tree.

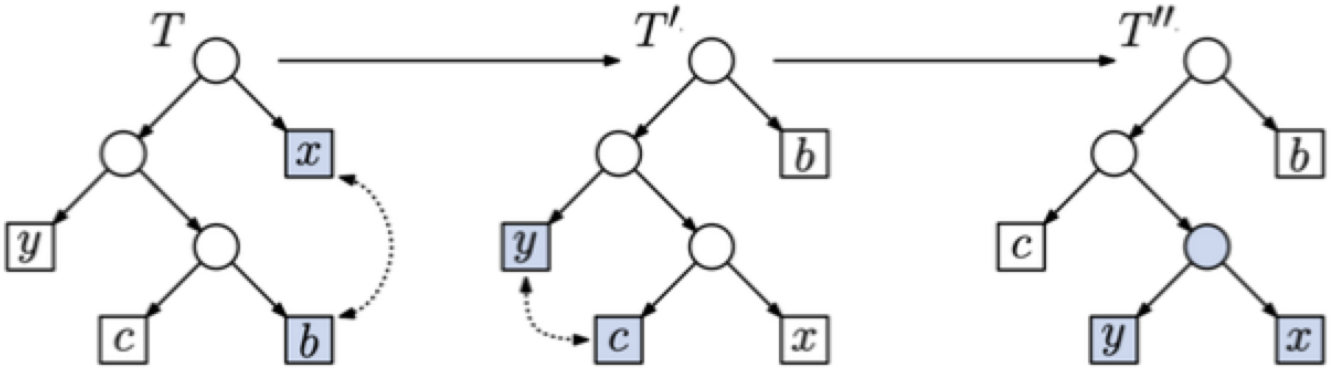
Time Complexity: $O(n \log n)$ for $n - 1$ extractions and insertions in the priority queue.

Optimality of Huffman Tree

In any optimal Huffman Tree, the two characters x and y with the smallest probabilities are siblings at the maximum depth.

Proof Outline:

- Let T be an optimal prefix code tree with siblings b and c at the maximum depth.
- If $\{x, y\} = \{b, c\}$, then x and y are at the maximum depth, so we are done.
- Otherwise, assume $p(b) \leq p(c)$ and $p(x) \leq p(y)$.
  - Since x and y have the smallest probabilities, $p(x) \leq p(b)$ and $p(y) \leq p(c)$.
  - As b and c are at maximum depth, $d_T(b) \geq d_T(x)$ and $d_T(c) \geq d_T(y)$.



Optimality of Huffman Tree

Switching Argument: To show optimality, suppose we swap x with b, creating a new tree T'.

- Most nodes contribute the same to the cost in both T and T'. The only exceptions are x and b.
- Let $B(T)$ denote the total cost of T. We calculate the change in cost due to the swap as follows:

$$B(T') = B(T) - (p(x)d_T(x) + p(b)d_T(b)) + (p(x)d_T(b) + p(b)d_T(x))$$

- Simplifying this, we get:

$$B(T') = B(T) - (p(b) - p(x))(d_T(b) - d_T(x))$$

- Since $(p(b) - p(x))(d_T(b) - d_T(x)) \geq 0$, it follows that $B(T') \leq B(T)$.

Similar we get $B(T'') \leq B(T')$.

Optimality of Huffman's Algorithm

Theorem: Huffman's algorithm constructs an optimal prefix code tree.

Proof (by Induction on n):

- **Base Case:** For $n = 1$, only one tree is possible, which is trivially optimal.
- **Inductive Step:** Assume Huffman's algorithm is optimal for $n - 1$ characters.
  - For $n \geq 2$, let x and y be the two lowest-probability characters.
  - Huffman's algorithm combines x and y into a new node z with $p(z) = p(x) + p(y)$, reducing the alphabet to $n - 1$.
  - By the hypothesis, the resulting tree T_{n-1} is optimal.
- **Construct T_n:** Replace z with x and y in T_{n-1}, adding $p(z)$ to the total cost.
- Since T_{n-1} is optimal and this added cost is fixed, T_n is also optimal.

Optimal File Merging Problem

You are given a list of files, each with a positive size. Merging two files of sizes a and b costs $a + b$. Your goal is to merge all files into a single file with the minimum total cost.

Example:

- Given file sizes: $[20, 4, 8, 2]$.
- Optimal merging sequence:
  - Merge 2 and 4 (cost $= 6$).
  - Merge 6 (from previous) and 8 (cost $= 14$).
  - Merge 14 (from previous) and 20 (cost $= 34$).
- Total cost $= 6 + 14 + 34 = 54$.

Solution: Greedy Algorithm using Min-Heap

Algorithm Steps:

- Initialize a **min-heap** (priority queue) with the sizes of all files.
- While more than one file remains in the heap:
  - Extract the two smallest files from the heap.
  - Calculate the merge cost as the sum of these two files.
  - Add this cost to a running total.
  - Insert the merged file (with the new size) back into the heap.
- When one file remains, the running total is the minimum merge cost.

Complexity:

- Each merge operation involves $O(\log n)$ for heap insertion and extraction.
- Total time complexity: $O(n \log n)$, where n is the number of files.

Prim's Algorithm for Minimum Spanning Trees (MST)

- **Algorithm Overview**:
  - Start from any vertex $v \in V$ and grow the MST.
  - At each step, add the minimum-cost edge that connects a vertex in the MST to a vertex outside it.
  - Repeat until all vertices are included in the MST.
- **Implementation and Complexity**:
  - Using a priority queue (binary heap), total runtime: $O(m \log m)$.
  - Each edge is processed once, and the priority queue efficiently manages minimum edge selection.
- **Exchange Argument for Correctness**:
  - Assume an optimal MST T^* differs from the MST T constructed by Prim's. Find the first edge in T that isn't in T^*. Replacing the corresponding edge in T^* with Prim's edge cannot increase the MST cost.
  - Repeat this swapping process until $T = T^*$

Kruskal's Algorithm for Minimum Spanning Trees (MST)

- **Algorithm Overview**:
  - Sort all edges by weight, then process each edge in ascending order.
  - Add each edge to the MST unless it creates a cycle.
  - Use a disjoint-set data structure to efficiently check if two nodes are connected.
- **Implementation and Complexity**:
  - Sorting edges takes $O(m \log m)$; disjoint-set operations take $O(m\alpha(m))$, where $\alpha(m)$ is the inverse Ackermann function.
  - Overall runtime: $O(m \log m)$.
- **Exchange Argument for Correctness**:
  - Assume Kruskal's MST T differs from an optimal MST T^*. For the first edge in T not in T^*, add it to T^* and remove a higher-cost edge to form a new MST.
  - This process gradually transforms T^* into T, preserving the minimum weight

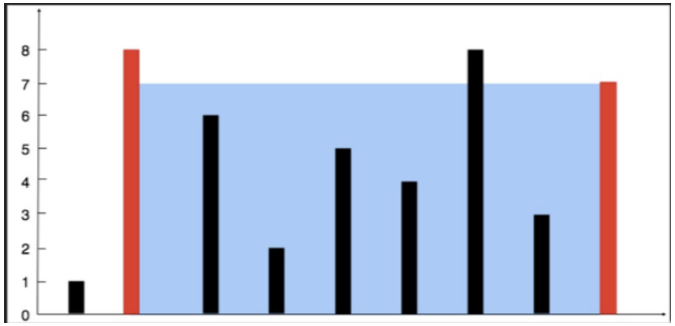
Container With Most Water

Problem:

Given an integer array `height` of length n, representing vertical lines at points $(i, 0)$ to $(i, \texttt{height}[i])$. Find two lines that, together with the x-axis, form a container that holds the maximum possible amount of water.

The container's area between lines at indices i and j is:

$$\text{Area} = \min(\text{height[i], height[j]}) \times (j - i)$$



Example:

```
Input: height = [1, 8, 6, 2, 5, 4, 8, 3, 7]
Output: 49
```

Maximum Water Container

Two pointer Solution

1. Initialize `left` at start, `right` at end of `height`.
2. Compute area:

$$\text{Area} = \min(\text{height[left]}, \text{height[right]}) \times (\textit{right} - \textit{left})$$

3. Update `max_area` if area is larger.
4. Move the pointer with the smaller height:
   - If `height[left] < height[right]`, increment `left`.
   - Else, decrement `right`.

Complexity: $O(n)$ time, $O(1)$ space

Example: `height = [1, 8, 6, 2, 5, 4, 8, 3, 7]`
- Max Area: 49, with `left = 1`, `right = 8`

Coin Change Problem (Again)

Problem Statement:

Given a set of coin denominations $\{1, 5, 10, 25\}$ and a target amount M, determine the minimum number of coins needed to make the amount M.

Example:

If $M = 63$, the minimum number of coins needed is 6:

$$63 = 25 + 25 + 10 + 1 + 1 + 1$$

using two 25-cent coins, one 10-cent coin, and three 1-cent coins.

Greedy Algorithm for Coin Change

Algorithm Steps:

- Sort the coins in descending order, $\{25, 10, 5, 1\}$.
- For each coin c_i in this order:
  - While $M \geq c_i$, take a coin of value c_i.
  - Subtract c_i from M and increase the coin count.
- Repeat until $M = 0$ or no more coins are available.

Greedy works in this case!!

Job Sequencing with Deadlines Problem:

You are given a list of n jobs, each with:

- A **profit** if the job is completed by its **deadline**.
- A **deadline**, indicating the latest time by which the job can be completed. Each job takes exactly one unit of time and only one job can be scheduled at any given time slot. The objective is to find a schedule for the jobs such that the total profit is maximized.

Example:

- Jobs with profits and deadlines: $[(A, 100, 2), (B, 19, 1), (C, 27, 2), (D, 25, 1), (E, 15, 3)]$.
- Optimal schedule: $C \to A \to E$ (profits 27, 100, 15).
- Total profit $= 27 + 100 + 15 = 142$.

Greedy Algorithm

Algorithm

- **Step 1: Sort Jobs by Profit.**
  - Sort all jobs in descending order of profit to prioritize higher profits.
- **Step 2: Initialize Time Slots.**
  - Create an array of available slots, initialized to indicate all slots are free.
- **Step 3: Assign Jobs to Latest Possible Slot.**
  - For each job, find the latest available slot from its deadline and move backward.
  - If a slot is available, assign the job to this slot and add the profit to the total.
  - If no slot is available, skip the job.

Minimum Number of Platforms Problem

Given train **arrival** and **departure** times, find the minimum number of platforms needed so no train waits.

(Greedy Algorithm):

- Sort arrival and departure times separately.
- Use two pointers to traverse both arrays:
  - Increment platform count if an arrival comes before the next departure.
  - Decrement platform count if a departure comes before the next arrival.
- Track the maximum platforms needed at any point.

Interval Covering Problem

Given a target interval $[T_{\text{start}}, T_{\text{end}}]$ and a set of intervals, find the minimum number of intervals needed to fully cover $[T_{\text{start}}, T_{\text{end}}]$.

Greedy Algorithm:

- Sort intervals by start time.
- Initialize `current_end` at T_{start}.
- For each interval, select the one extending coverage the farthest:
  - Update `current_end` to the farthest end within overlapping intervals.
  - Increment count and continue until `current_end` $\geq T_{\text{end}}$.
- If no interval can extend `current_end`, return -1 (target not fully covered).